

Frontend Developer Training Guide

A complete roadmap from fundamentals to advanced concepts for frontend development.

Table of Contents

1. [Internet & Web Fundamentals](#)
 2. [HTML — Structure of the Web](#)
 3. [CSS — Styling the Web](#)
 4. [JavaScript — Core Language](#)
 5. [JavaScript — Advanced Concepts](#)
 6. [Version Control — Git & GitHub](#)
 7. [Package Managers & Build Tools](#)
 8. [React.js — Library Fundamentals](#)
 9. [React.js — Advanced Patterns](#)
 10. [State Management](#)
 11. [TypeScript](#)
 12. [API Communication](#)
 13. [CSS Frameworks & Preprocessors](#)
 14. [Testing](#)
 15. [Performance Optimization](#)
 16. [Web Security](#)
 17. [Accessibility \(a11y\)](#)
 18. [Progressive Web Apps \(PWA\)](#)
 19. [DevTools & Debugging](#)
 20. [Deployment & CI/CD](#)
 21. [Soft Skills & Best Practices](#)
-

1. Internet & Web Fundamentals

Learning Objective

Understand how the internet works, including the client-server model, how data travels across networks, and how browsers retrieve and render web pages. Learn the role of key protocols such as HTTP/HTTPS and DNS. By the end, you should be able to explain what happens — technically — from the moment a user types a URL to the moment a page appears on screen.

Curriculum

1. [How the Internet Works — MDN](#)
 2. [How the Web Works — MDN](#)
 3. [What is HTTP? — MDN](#)
 4. [What is a Domain Name? — MDN](#)
 5. [What is a URL? — MDN](#)
 6. [How Browsers Work — web.dev](#)
 7. [Critical Rendering Path — web.dev](#)
 8. [HTTPS and Why It Matters — web.dev](#)
-

Practice Exercises

Exercise 1: Trace a Web Request End-to-End

Write a step-by-step technical narrative (minimum 10 steps) documenting exactly what happens when a user types `https://www.example.com` into a browser and presses Enter. It must include: - URL parsing and browser cache checks - DNS resolution (recursive resolver, root nameserver, TLD nameserver, authoritative nameserver) - TCP three-way handshake - TLS/SSL handshake for HTTPS - HTTP request structure (method, headers, body) - Server processing and response formation - Browser response handling (HTML parsing, sub-resource discovery, rendering) - A simple text-based diagram illustrating the full client-server round trip

Exercise 2: HTTP Methods & Status Codes Reference Sheet

Build a personal reference sheet (`.md` or `.html` file) that includes: - A table of the five primary HTTP methods (GET, POST, PUT, PATCH, DELETE) with descriptions and real-world examples - At least 12 HTTP status codes (covering 2xx, 3xx, 4xx, 5xx ranges), each with a description and scenario - A section explaining HTTP vs HTTPS and what a certificate authority (CA) is - A section explaining stateless vs stateful communication and how cookies/sessions/tokens work around HTTP's statelessness

Exercise 3: Browser Rendering Walkthrough

Using Chrome DevTools, analyze a real webpage and produce a written report that includes: - A screenshot of the Network tab waterfall, identifying render-blocking resources - A description of each rendering pipeline step (DOM, CSSOM, render tree, layout, paint, compositing) as applied to the page - At least three observations about how the page could load faster - An explanation of DNS prefetching, preloading, and lazy loading — and whether the page uses any of them

Acceptance Criteria

- ☐ Can verbally explain the client-server model without notes
 - ☐ Can describe the full lifecycle of an HTTP request-response cycle
 - ☐ Can explain the difference between HTTP and HTTPS
 - ☐ Can explain what DNS does and how it resolves a domain to an IP
 - ☐ Can name all 8 steps of the browser rendering pipeline in correct order
 - ☐ Can define: URL, Domain, Hosting, Browser, Protocol, TCP/IP, TLS
 - ☐ Exercise 1 — narrative is complete and technically accurate
 - ☐ Exercise 2 — reference sheet has all required tables and sections
 - ☐ Exercise 3 — report includes DevTools screenshots and all required analysis
-

2. HTML — Structure of the Web

Learning Objective

Learn how to write well-structured, semantically meaningful HTML that forms the backbone of any web page. Understand the purpose of each core element, how forms collect user input, and why semantic markup matters for accessibility and SEO. By the end, you should be able to build a complete, standards-compliant HTML document from scratch.

Curriculum

1. [HTML Introduction — W3Schools](#)
 2. [HTML Basic — W3Schools](#)
 3. [HTML Elements — W3Schools](#)
 4. [HTML Attributes — W3Schools](#)
 5. [HTML Headings — W3Schools](#)
 6. [HTML Paragraphs — W3Schools](#)
 7. [HTML Links — W3Schools](#)
 8. [HTML Images — W3Schools](#)
 9. [HTML Lists — W3Schools](#)
 10. [HTML Tables — W3Schools](#)
 11. [HTML Forms — W3Schools](#)
 12. [HTML Form Input Types — W3Schools](#)
 13. [HTML Semantic Elements — W3Schools](#)
 14. [HTML Head — W3Schools](#)
 15. [HTML Accessibility — W3Schools](#)
-

Practice Exercises

Exercise 1: Build a Complete Personal Profile Page

Create a fully structured HTML-only page (no CSS, no JavaScript) representing a personal profile. It must include: - Proper `<!DOCTYPE html>`, `<html lang="en">`, `<head>` with charset, viewport meta, descriptive `<title>`, and at least two additional meta tags (description + one Open Graph tag) - `<header>` with heading, subtitle, and `<nav>` with unordered list of navigation links - `<main>` with at least four `<section>` elements: About Me, Skills, Work Experience (table with `<thead>`, `<tbody>`, `<tfoot>`, at least 3 rows and 4 columns), and Contact - An `<aside>` with supplementary content (quote or link list) - At least one image with `alt` text wrapped in `<figure>` / `<figcaption>` - An ordered list and an unordered list used in semantically appropriate contexts - A `<footer>` with copyright text and a link

Exercise 2: Build a Multi-Field Registration Form

Create a standalone HTML page with a comprehensive registration form that includes: - At least two `<fieldset>` groups with `<legend>` (e.g., Personal Info and Account Setup) - Text inputs for first name, last name, username — each with `<label>` associated via `for / id` - Email input, password input with `minlength`, and `tel` input with `pattern` - `<select>` dropdown with at least five options including a disabled placeholder - A group of at least three radio buttons sharing the same `name` - At least two checkboxes with labels (terms, newsletter consent) - `<textarea>` with `rows`, `cols`, and `placeholder` - A date input and a range input with `min`, `max`, `step` - File upload input accepting only images - Submit button and reset button - All required fields marked with `required` attribute

Exercise 3: Audit and Rewrite a Non-Semantic Page

Write a “bad” HTML page using only `<div>` and `<span>` (no semantic meaning), then rewrite it as a properly semantic version: - The “bad” version: 20-30 lines using `<div class="header">`, `<div class="nav">`, `<div class="article">`, etc. - The “good” version: using `<header>`, `<nav>`, `<main>`, `<article>`, `<section>`, `<aside>`, `<footer>`, `<figure>`, etc. - Written explanation documenting every change and why, referencing benefits for: (a) screen reader users, (b) search engines, (c) developers reading the code - Validate both versions at [W3C Markup Validator](#) and note any errors

Acceptance Criteria

- ☐ Can write a valid HTML document structure from memory
- ☐ Can correctly use heading hierarchy (`<h1>` through `<h6>`)
- ☐ Can build a table with `<thead>`, `<tbody>`, `<tfoot>`, `colspan`, and `rowspan`
- ☐ Can build a form with at least 8 different input types and proper `<label>` associations
- ☐ Can explain the difference between `<div>` and semantic elements like `<article>`, `<section>`, `<aside>`
- ☐ Can explain what `alt` does and why it is required for accessibility
- ☐ Can explain meta tags and how viewport meta affects behavior
- ☐ Exercise 1 — profile page passes W3C validation with zero errors
- ☐ Exercise 2 — form is complete with all required input types, fieldsets, and labels
- ☐ Exercise 3 — includes both versions, written audit, and validator results

3. CSS — Styling the Web

Learning Objective

Learn how to apply visual styling to HTML documents using CSS, from basic selectors and the box model through Flexbox, Grid, responsive design, and animations. Understand how the browser resolves styling conflicts through specificity and the cascade. By the end, you should be able to build a fully styled, responsive webpage layout using only CSS.

Curriculum

1. [CSS Introduction — W3Schools](#)
2. [CSS Selectors — W3Schools](#)
3. [CSS Specificity — W3Schools](#)
4. [CSS Box Model — W3Schools](#)
5. [CSS Display — W3Schools](#)
6. [CSS Position — W3Schools](#)
7. [CSS Flexbox — W3Schools](#)
8. [CSS Flex Container — W3Schools](#)
9. [CSS Flex Items — W3Schools](#)
10. [CSS Grid — W3Schools](#)
11. [CSS Grid Container — W3Schools](#)
12. [CSS Responsive Design — W3Schools](#)
13. [CSS Media Queries — W3Schools](#)
14. [CSS Variables — W3Schools](#)
15. [CSS Transitions — W3Schools](#)
16. [CSS Animations — W3Schools](#)

- 17. [CSS Pseudo-classes — W3Schools](#)
- 18. [CSS Pseudo-elements — W3Schools](#)
- 19. [Flexbox Froggy \(Interactive Game\)](#)
- 20. [Grid Garden \(Interactive Game\)](#)

Practice Exercises

Exercise 1: Flexbox + Grid Dashboard Layout

Build a responsive admin dashboard page using only HTML and CSS (no frameworks, no JS). It must include: - A fixed top navbar built with Flexbox: logo left, nav links centered, user icon right — stays visible on scroll - A two-column layout with CSS Grid: sidebar (fixed width) + main content area — collapses to single column below 768px - Inside main area, a card grid using `repeat(auto-fill, minmax(220px, 1fr))` with at least six cards - Each card uses Flexbox internally to arrange icon, title, value, and link in a column layout - A footer with Flexbox: three columns (copyright, links, social) — single centered column on mobile - CSS variables in `:root` for all colors, font sizes, spacing, and border-radius - At least one CSS transition (card hover) and one `@keyframes` animation (fade-in or pulse)

Exercise 2: Responsive Marketing Landing Page

Build a fully responsive landing page from scratch (HTML + CSS only). Must look polished on mobile (375px) and desktop (1280px): - Hero section with full-width background, centered headline + subheadline, two CTA buttons (filled + outlined) with hover transitions - Features section: three-column CSS Grid on desktop, single column on mobile, each with icon, heading, description - Testimonials section: Flexbox horizontal layout on desktop, vertical on mobile, at least three cards - Pricing section: three tier cards, middle "recommended" card visually emphasized with CSS only - Sticky nav with smooth scroll (`scroll-behavior: smooth`) linking to sections by ID anchor - Contact form with styled focus states using CSS variable colors - Mobile-first media queries with at least three breakpoints (480px, 768px, 1024px)

Exercise 3: Complete Flexbox Froggy + Grid Garden

Complete these interactive games to reinforce layout concepts: - [Flexbox Froggy](#) — complete all 24 levels - [Grid Garden](#) — complete all 28 levels - Take a screenshot of each completion screen

Acceptance Criteria

- ☐ Can write CSS using all selector types without looking them up
- ☐ Can calculate specificity score of any CSS rule in `(a, b, c)` notation
- ☐ Can explain box model and `box-sizing: border-box` vs `content-box`
- ☐ Can explain the difference between all five position values
- ☐ Can build a horizontal navbar with Flexbox from memory
- ☐ Can build a responsive card grid with CSS Grid using `auto-fill` and `minmax`
- ☐ Can explain when to use Flexbox vs Grid
- ☐ Can write media queries targeting specific screen width ranges
- ☐ Can define and use CSS custom properties across a stylesheet
- ☐ Exercise 1 — dashboard is fully responsive, uses both Flexbox and Grid
- ☐ Exercise 2 — landing page is polished, responsive, mobile-first
- ☐ Exercise 3 — screenshots of Flexbox Froggy (level 24) and Grid Garden (level 28)

4. JavaScript — Core Language

Learning Objective

Learn JavaScript from the ground up — syntax, data types, control structures, and built-in methods. Understand `var / let / const`, how functions work in multiple forms, and how to work with arrays and objects. By the end, you should be able to write functional JavaScript programs that solve real problems using core language features.

Curriculum

1. [JavaScript Introduction — W3Schools](#)
 2. [JavaScript Variables — W3Schools](#)
 3. [JavaScript let — W3Schools](#)
 4. [JavaScript const — W3Schools](#)
 5. [JavaScript Data Types — W3Schools](#)
 6. [JavaScript Operators — W3Schools](#)
 7. [JavaScript Comparisons — W3Schools](#)
 8. [JavaScript Conditions — W3Schools](#)
 9. [JavaScript For Loop — W3Schools](#)
 10. [JavaScript Functions — W3Schools](#)
 11. [JavaScript Arrow Functions — W3Schools](#)
 12. [JavaScript Strings — W3Schools](#)
 13. [JavaScript String Methods — W3Schools](#)
 14. [JavaScript Arrays — W3Schools](#)
 15. [JavaScript Array Methods — W3Schools](#)
 16. [Array Methods — javascript.info](#)
 17. [JavaScript Objects — W3Schools](#)
 18. [Error Handling — javascript.info](#)
-

Practice Exercises

Exercise 1: Data Transformation Pipeline

Write a JavaScript program that processes an array of at least 10 fictional employee objects (each with `id`, `name`, `department`, `salary`, `yearsExperience`). It must:

- Use `filter()` to extract employees in a specific department
- Use `map()` to transform them into objects with `name` and adjusted salary (10% raise)
- Use `reduce()` to calculate the total salary budget for all employees
- Use `sort()` to sort by salary descending
- Use `find()` to locate a specific employee by `id`
- Use `some()` and `every()` to answer two questions (e.g., "Anyone earn over \$100k?", "Everyone has >1 year experience?")
- Wrap in `try/catch` handling `null / undefined` input
- Log each result with a descriptive label

Exercise 2: String Utility Library

Write a `StringUtils` module with at least eight functions:

- `capitalize(str)` — first letter uppercase, rest lowercase
- `truncate(str, maxLength)` — truncate with `...` if exceeds limit; throw `TypeError` if not a string
- `countWords(str)` — count words, ignoring extra whitespace
- `isPalindrome(str)` — case-insensitive, ignore spaces/punctuation
- `toSlug(str)` — convert "Hello World! Title" to "hello-world-title"
- `extractEmails(str)` — extract all emails using regex
- `reverseWords(str)` — reverse word order
- `maskSensitive(str, visibleChars)` — mask all but last N chars with `*`

- Write at least two `console.assert()` test cases per function

Exercise 3: Mini Shopping Cart System

Build a pure JavaScript shopping cart (no DOM). Must include: - `products` array with at least eight product objects (`id` , `name` , `price` , `category` , `inStock`) - `addToCart(productId, quantity)` — add or increment; throw error if out of stock - `removeFromCart(productId)` — remove item entirely - `updateQuantity(productId, newQuantity)` — update quantity; remove if 0 - `getCartSummary()` — return `{ items, totalItems, subtotal, tax (8%), total }` rounded to 2 decimals - `clearCart()` and `applyDiscount(code)` — apply percentage from a predefined code map - `filterProducts(category)` — return in-stock products in a category - Call all functions in sequence, logging cart summary after each operation

Acceptance Criteria

- ☐ Can explain the difference between `var` , `let` , and `const`
- ☐ Can name all seven JavaScript data types with examples
- ☐ Can explain `==` vs `===` and why strict equality is preferred
- ☐ Can write functions using both declaration and arrow syntax
- ☐ Can use `map` , `filter` , `reduce` , `find` , `some` , `every` , `sort` without looking up syntax
- ☐ Can use `Object.keys()` , `Object.values()` , `Object.entries()`
- ☐ Can write `try/catch/finally` and explain when each block runs
- ☐ Can use template literals, destructuring, and spread operator
- ☐ Exercise 1 — runs without errors, produces correct labeled output
- ☐ Exercise 2 — all `console.assert()` tests pass
- ☐ Exercise 3 — handles all edge cases, logs correct summaries

5. JavaScript — Advanced Concepts

Learning Objective

Build on core JavaScript to understand closures, `this` , asynchronous operations (event loop, Promises, `async/await`), DOM manipulation, and browser storage. By the end, you should be able to write async JavaScript, manipulate the DOM dynamically, and explain JavaScript's concurrency model.

Curriculum

1. [Closures — javascript.info](#)
2. [Prototypal Inheritance — javascript.info](#)
3. [Classes — javascript.info](#)
4. [this Keyword — javascript.info](#)
5. [Arrow Functions and this — javascript.info](#)
6. [Callbacks — javascript.info](#)
7. [Promises — javascript.info](#)
8. [Promise Chaining — javascript.info](#)
9. [Async/Await — javascript.info](#)
10. [Event Loop — javascript.info](#)
11. [Microtasks — javascript.info](#)
12. [Destructuring — javascript.info](#)
13. [Rest/Spread — javascript.info](#)

[14. DOM Introduction — javascript.info](#)[15. DOM Events — javascript.info](#)[16. LocalStorage — javascript.info](#)

Practice Exercises

Exercise 1: Closure Deep Dive — Counter Factory & Memoization

Write JavaScript that demonstrates genuine understanding of closures: - A `makeCounter(start, step)` factory returning `{ increment(), decrement(), reset(), getValue() }` — internal count must not be accessible from outside - Create three independent counter instances with different `start / step` values; demonstrate they maintain independent state - A `memoize(fn)` higher-order function that returns a memoized version using a private cache; log cache hit/miss - Demonstrate `memoize` wrapping a Fibonacci function — show repeated calls return cached results - Written comment explaining: (a) what a closure is, (b) what lexical scope means, (c) why private state can't be accessed externally

Exercise 2: Async JavaScript — Fetch, Promises, and Async/Await

Build a program that fetches data from `https://jsonplaceholder.typicode.com/` using both patterns: - `fetchUser(userId)` using `.then().catch()` Promise chain syntax - `fetchPostsByUser(userId)` using `async/await` with `try/catch` - `getUserWithPosts(userId)` combining both into `{ user, posts, postCount }` - Demonstrate `Promise.all()` fetching three users simultaneously - Demonstrate `Promise.allSettled()` with one intentionally bad URL — log which succeeded/failed - Build a `sleep(ms)` utility with `new Promise(resolve => setTimeout(resolve, ms))` and use it in an async function - Written comment explaining the difference between `Promise.all()` and `Promise.allSettled()`

Exercise 3: DOM Task Manager

Build a fully functional interactive task manager with vanilla HTML, CSS, and JavaScript (no libraries): - Text input + "Add Task" button that appends tasks to a list (also works on Enter keypress); input clears and refocuses after submission - Each task shows: text, timestamp, "Complete" button (toggles strikethrough), "Delete" button (removes with CSS fade-out) - Filter bar: "All", "Active", "Completed" buttons that show/hide tasks via DOM manipulation - Live counter showing `X tasks remaining` — updates on add/complete/delete - All tasks persisted to `localStorage` — survive page refresh with correct completed state - Event delegation: single listener on the list container handles all Complete/Delete clicks - "Clear Completed" button removes all completed tasks from DOM and `localStorage`

Acceptance Criteria

- ☐ Can explain what a closure is and give a practical use case
- ☐ Can explain what `this` refers to in: regular function, arrow function, class method, event handler
- ☐ Can explain the event loop, call stack, task queue, and microtask queue
- ☐ Can predict execution order of mixed sync/async code snippets
- ☐ Can write Promise chains and equivalent async/await with proper error handling
- ☐ Can explain `Promise.all()` vs `Promise.allSettled()`
- ☐ Can use `querySelector`, `createElement`, `classList`, `addEventListener` confidently
- ☐ Can explain `localStorage` vs `sessionStorage` persistence behavior
- ☐ Can explain event delegation and why it's preferred
- ☐ Exercise 1 — closures demo runs correctly with independent counters and memoization
- ☐ Exercise 2 — fetches real API data, handles errors, demonstrates both patterns
- ☐ Exercise 3 — task manager is fully functional, persists data, uses event delegation

6. Version Control — Git & GitHub

Learning Objective

Learn Git and GitHub for managing code history, collaborating through branches and pull requests, and following professional workflows. By the end, you should be comfortable with the full GitHub collaboration cycle — from forking to getting a PR reviewed and merged — and understand commit message conventions and Git Flow branching.

Curriculum

1. [Git Basics — git-scm.com](#)
 2. [First-Time Git Setup](#)
 3. [Git Commands Reference](#)
 4. [Branching and Merging](#)
 5. [Viewing History](#)
 6. [Undoing Changes](#)
 7. [Git Flow Branching Model](#)
 8. [Conventional Commits](#)
 9. [GitHub Pull Requests](#)
 10. [Interactive Git Branching Practice](#)
-

Practice Exercises

Exercise 1: Local Repository and Daily Workflow

Set up a Git repo for a fictional project called `intern-portfolio` : - Initialize with `git init` , create `README.md` , `index.html` , `style.css` - Stage files individually and verify with `git status` and `git diff --staged` - Write at least 5 commits following Conventional Commits format: `feat:` , `fix:` , `docs:` , etc. - View history with `git log --oneline --graph` and include output in submission

Exercise 2: Branching, Merging, and Conflict Resolution

Simulate a team branching workflow: - Create `develop` from `main` , then two feature branches: `feature/navbar` and `feature/footer` - Make changes to the same file on both branches to intentionally cause a merge conflict - Merge `feature/navbar` into `develop` first, then merge `feature/footer` and manually resolve the conflict - Tag the final commit with `git tag v0.1.0`

Exercise 3: GitHub Collaboration — Fork, PR, and Review

Practice the full GitHub PR workflow: - Fork a training repo, clone locally - Create branch `feat/your-name-profile-card` , add an HTML profile card to `profiles/` - Push to your fork, open a Pull Request with descriptive title and body - Respond to at least one round of review feedback with code changes

Acceptance Criteria

- ☐ Repo has at least 5 commits following Conventional Commits format
- ☐ `git log --oneline --graph` shows clear history
- ☐ Feature branches created from `develop` following Git Flow naming
- ☐ Merge conflict intentionally created and manually resolved

- ☐ Repo tagged with `v0.1.0`
 - ☐ PR opened on GitHub with descriptive title and body
 - ☐ At least one round of review feedback addressed
 - ☐ Can explain `git merge` vs `git rebase` verbally
-

7. Package Managers & Build Tools

Learning Objective

Understand how npm manages project dependencies and scripts. Learn to configure `package.json`, understand semantic versioning, set up Vite as a modern build tool, and enforce code quality with ESLint and Prettier.

Curriculum

1. [npm Getting Started](#)
 2. [npm CLI Commands](#)
 3. [package.json Reference](#)
 4. [Semantic Versioning](#)
 5. [Vite — Getting Started](#)
 6. [Vite Configuration](#)
 7. [ESLint — Getting Started](#)
 8. [Prettier Documentation](#)
 9. [Integrating Prettier with ESLint](#)
-

Practice Exercises

Exercise 1: Scaffold and Inspect a Project with npm

Create a project from scratch and understand every part of it: - Run `npm init` (not `-y`) and fill every prompt thoughtfully - Add a `scripts` section with: `start`, `build`, `lint`, `format` - Install two runtime deps (e.g., `lodash`, `axios`) and two dev deps (e.g., `eslint`, `prettier`) - Annotate the meaning of each version range symbol (`^`, `~`, exact) in a notes file - Delete `node_modules`, run `npm install`, verify project restores correctly

Exercise 2: Set Up Vite and Explore the Build

Scaffold a Vite project and understand the toolchain: - Run `npm create vite@latest my-vite-app -- --template vanilla` and explore every generated file - Start dev server, make a change, observe HMR updating instantly - Run `npm run build`, examine `dist/` folder — explain why content hashing is used - Add a custom path alias in `vite.config.js` and use it in an import - Run `npm run preview` and confirm production build matches dev

Exercise 3: Configure ESLint + Prettier

Set up code quality tools: - Install `eslint`, `prettier`, `eslint-config-prettier` - Initialize ESLint with `npx eslint --init` - Create `.prettierrc` with at least five rules (`printWidth`, `tabWidth`, `singleQuote`, `trailingComma`, `semi`) - Introduce three linting errors deliberately and fix them with `npx eslint .` - Add `lint` and `format` scripts to `package.json`, verify both run successfully

Acceptance Criteria

- ☐ `package.json` has all fields populated and four custom scripts
 - ☐ `dependencies` and `devDependencies` correctly separated
 - ☐ Can explain `^1.2.3` vs `~1.2.3` vs `1.2.3` without docs
 - ☐ Vite project runs with `npm run dev`
 - ☐ Production build generated and `dist/` contents examined
 - ☐ ESLint and Prettier configured without conflicts
 - ☐ Both `npm run lint` and `npm run format` execute without errors
-

8. React.js — Library Fundamentals

Learning Objective

Build a solid foundation in React by understanding component-based UI development, JSX syntax, and essential Hooks (`useState`, `useEffect`, `useRef`, `useMemo`, `useCallback`). By the end, you should be able to build a multi-component interactive application, apply conditional rendering, render dynamic lists with keys, and handle user events.

Curriculum

1. [What is React? — react.dev](#)
 2. [Installation](#)
 3. [Writing Markup with JSX](#)
 4. [JavaScript in JSX](#)
 5. [Your First Component](#)
 6. [Passing Props](#)
 7. [useState Hook](#)
 8. [useEffect Hook](#)
 9. [useRef Hook](#)
 10. [useMemo Hook](#)
 11. [useCallback Hook](#)
 12. [Responding to Events](#)
 13. [Conditional Rendering](#)
 14. [Rendering Lists](#)
-

Practice Exercises

Exercise 1: Component Architecture — Team Directory

Build a static but well-structured React app displaying a team directory: - Scaffold with `npm create vite@latest team-directory -- --template react` - Component hierarchy: `App` > `TeamSection` > `MemberCard` - `TeamSection` uses `children` prop to wrap `MemberCard` components - `MemberCard` accepts `name`, `role`, `avatarUrl`, optional `bio` with default value - At least two team groups (e.g., "Engineering", "Design") with 3+ members each - Data defined in a separate `data/team.js` file - CSS Modules (`.module.css`) for component-scoped styling

Exercise 2: Interactive State Management with Hooks

Build a task management mini-app demonstrating all five major hooks: - `useState` — manage task list (objects with `id`, `text`, `completed`, `priority`) and a search filter - `useRef` — auto-focus input on mount; store a non-re-rendering task counter - `useMemo` — compute filtered/sorted task list that only re-runs when tasks or search changes - `useCallback` — memoize `handleAddTask`, `handleToggleComplete`, `handleDeleteTask` - `useEffect` — persist tasks to `localStorage` on change; restore on mount

Exercise 3: Product Catalog with API Data

Build a product catalog fetching from `https://fakestoreapi.com/products` : - Fetch products on mount with `useEffect` + `fetch` - Four UI states: loading spinner, error message, empty state, product grid - Use ternary operator for one condition, `&&` for another, early return for another - Render `ProductCard` components with product `id` as key (not array index — comment explaining why) - Category filter bar reading unique categories from data — clicking filters products

Acceptance Criteria

- ☐ React app scaffolded with Vite and runs without errors
- ☐ Three levels of component nesting with props at each level
- ☐ `children` prop used in at least one wrapper component
- ☐ Default prop values using destructuring defaults
- ☐ `useState`, `useEffect`, `useRef`, `useMemo`, `useCallback` each used meaningfully
- ☐ `localStorage` read on mount and written on state change
- ☐ All three conditional rendering patterns used: ternary, `&&`, early return
- ☐ Dynamic list with stable unique `key` props (not indices)
- ☐ Can explain `useMemo` vs `useCallback` and why keys matter

9. React.js — Advanced Patterns

Learning Objective

Move beyond fundamentals to learn custom hooks, Context API, React Router, `useReducer`, performance optimization, and Error Boundaries. By the end, you should be able to architect a multi-page React application that is performant, maintainable, and resilient to runtime errors.

Curriculum

1. [Custom Hooks — react.dev](#)
2. [Context API](#)
3. [Scaling with Reducer + Context](#)
4. [useReducer Reference](#)
5. [React Router Tutorial \(v6\)](#)
6. [React.memo Reference](#)
7. [React.lazy + Suspense](#)
8. [Error Boundaries](#)

Practice Exercises

Exercise 1: Build Custom Hooks

Extract reusable logic into three custom hooks: - `useLocalStorage(key, initialValue)` — behaves like `useState` but persists to `localStorage` ; handles `JSON.parse` failures gracefully - `useFetch(url)` — returns `{ data, loading, error }` ; uses `AbortController` to cancel in-flight requests on cleanup - `useDebounce(value, delay)` — returns debounced version of value - Demonstrate each in a real UI component (theme toggle, API data display, search input) - Each hook in its own file under `hooks/` with a JSDoc comment

Exercise 2: Context, useReducer, and Multi-Page Routing

Build a multi-page app combining Context API, useReducer, and React Router v6: - Four routes: `/` (Home), `/about` , `/dashboard` (protected), `/login` - `AuthContext` with `user` , `login(username)` , `logout()` — protected route redirects to `/login` - `ThemeContext` persisting light/dark preference with `useLocalStorage` hook - Dashboard state using `useReducer` with at least three action types (`ADD_NOTE` , `DELETE_NOTE` , `EDIT_NOTE`) - `ErrorBoundary` at top level catching render errors with fallback UI - `NavLink` highlighting active route

Exercise 3: Performance Optimization

Profile and optimize an intentionally unoptimized React app: - Use React DevTools Profiler to record flamegraph of unnecessary re-renders - Wrap pure children in `React.memo` and verify they stop re-rendering - Apply `useCallback` to handlers passed as props to memoized children - Apply `useMemo` to expensive computation (sorting/filtering 1000+ items) - Split app into three lazy-loaded routes with `React.lazy` + `Suspense` - Verify in Network tab that chunks load only when routes are visited

Acceptance Criteria

- ☐ `useLocalStorage` handles JSON parse errors and persists correctly
- ☐ `useFetch` has loading/error/data states and AbortController cleanup
- ☐ Custom hooks in `hooks/` directory with JSDoc comments
- ☐ React Router v6 with four routes and working navigation
- ☐ Protected route redirects unauthenticated users
- ☐ `useReducer` with at least three action types
- ☐ `ErrorBoundary` catches errors and shows fallback UI
- ☐ React DevTools Profiler used before/after optimization
- ☐ `React.memo` , `useCallback` , `useMemo` applied and verified
- ☐ `React.lazy` + `Suspense` used for route-level code splitting
- ☐ Can explain `React.memo` vs `useMemo` vs `useCallback`

10. State Management

Learning Objective

Develop a clear mental model for choosing the right state management solution. Learn Zustand for lightweight global state and TanStack Query for server state (data fetching, caching, mutations). Understand why using a single global solution for all state is an anti-pattern.

Curriculum

1. [Managing State — react.dev](#)

2. [Zustand — Getting Started](#)
 3. [Zustand — Slices Pattern](#)
 4. [Zustand — Persisting Store Data](#)
 5. [TanStack Query — Overview](#)
 6. [TanStack Query — Quick Start](#)
 7. [useQuery Reference](#)
 8. [useMutation Reference](#)
 9. [Query Invalidation](#)
-

Practice Exercises

Exercise 1: State Architecture Planning

Before writing code, complete a state-mapping exercise for a mid-complexity app (e.g., e-commerce storefront):

- Document the full component tree (at least 12 components, three nesting levels)
- For each state, assign to one of six categories: component-level, shared (lifted), global UI, server/remote, URL/navigation, form state
- Create a table: State Name | Category | Reason | Tool to Use
- Identify one case where global state was tempting but component-level was sufficient — explain why
- Review table with mentor before implementation

Exercise 2: Global UI State with Zustand

Build a Zustand store managing global UI state:

- Create `store/useAppStore.js` with two slices: `uiSlice ( theme , sidebarOpen , activeModal )` and `cartSlice ( addItem , removeItem , clearCart , computed totalPrice )`
- Persist theme to `localStorage` using Zustand's `persist` middleware — verify restoration on refresh
- Subscribe from two unrelated components (e.g., `CartIcon` in navbar and `CartDrawer`) — both update when cart changes
- Use `useShallow` selector to prevent unnecessary re-renders
- Add `reset` action to each slice; wire a “Reset App State” button

Exercise 3: Server State with TanStack Query

Replace manual `useEffect` data fetching with TanStack Query:

- Set up `QueryClient` + `QueryClientProvider` at root; enable React Query DevTools
- Replace at least three `useEffect` + `fetch` patterns with `useQuery` — configure meaningful `staleTime` values
- Implement `useMutation` for creating an item — on success, `invalidateQueries` to auto-refetch the list
- Implement optimistic updates for a toggle action using `onMutate`, `onError`, `onSettled`
- Add a dependent query using the `enabled` option (e.g., fetch user first, then their posts)

Acceptance Criteria

- ☐ State categorization table completed and reviewed with mentor
 - ☐ Zustand store with two slices using slices pattern
 - ☐ `persist` middleware applied and verified across page refresh
 - ☐ Two sibling components subscribing to the same Zustand store
 - ☐ `useShallow` used to prevent unnecessary re-renders
 - ☐ TanStack Query `QueryClient` set up with DevTools
 - ☐ At least three queries migrated from `useEffect` to `useQuery`
 - ☐ `useMutation` with `invalidateQueries` on success
 - ☐ Optimistic updates with rollback logic
 - ☐ Dependent query using `enabled` option
 - ☐ Can explain why server state should not live in Zustand
-

11. TypeScript

Learning Objective

Learn TypeScript as a typed superset of JavaScript that adds static type safety to React applications. By the end, you should be comfortable typing component props, hook return values, event handlers, API responses, and generic utilities.

Curriculum

1. [TypeScript in 5 Minutes](#)
2. [Everyday Types](#)
3. [Object Types \(Interfaces\)](#)
4. [Functions](#)
5. [Generics](#)
6. [Utility Types](#)
7. [TypeScript with React — react.dev](#)
8. [TypeScript Playground](#)

Practice Exercises

Exercise 1: TypeScript Fundamentals

Practice core type system features in plain `.ts` files: - Set up a project with `tsc --init` and `"strict": true` - Define variables with explicit types: `string`, `number`, `boolean`, `string[]`, tuple - Create an `interface User` with `id`, `name`, `email`, `role` (union: `"admin" | "editor" | "viewer"`), optional `avatarUrl?`, and `readonly createdAt` - Write a generic `getFirst<T>(arr: T[]): T | undefined` function and a generic `ApiResponse<T>` type - Practice utility types: `Partial<User>`, `Required<User>`, `Pick<User, "id" | "username">`, `Omit<User, "id">`

Exercise 2: Typing a React Application

Migrate a JavaScript React app to TypeScript: - Rename `.jsx` to `.tsx`, `.js` to `.ts`; install `@types/react`, `@types/react-dom` - Define `interface` or `type` for every component's props — no `any` anywhere - Type all `useState` calls explicitly where needed (e.g., `useState<User | null>(null)`) - Type all `useRef` with correct DOM element generic (e.g., `useRef<HTMLInputElement>(null)`) - Type all event handlers: `React.ChangeEvent<HTMLInputElement>`, `React.FormEvent<HTMLFormElement>`, etc.

Exercise 3: Advanced TypeScript Patterns

Apply advanced patterns for production-quality code: - Write a generic `useFetch<T>(url: string): { data: T | null; loading: boolean; error: string | null }` hook — no `any` internally - Build a type-safe `Select<T extends string>` component with generic `options`, `value`, and `onChange` - Use discriminated union for UI state: `{ status: "idle" } | { status: "loading" } | { status: "success"; data: Product[] } | { status: "error"; message: string }` - Write `getProperty<T, K extends keyof T>(obj: T, key: K): T[K]` with correct type inference

Acceptance Criteria

- ☐ TypeScript configured with `"strict": true`
- ☐ `interface` and `type` alias both used correctly
- ☐ Generic functions and types used with multiple concrete types
- ☐ `Partial`, `Required`, `Pick`, `Omit` utility types exercised

- ☐ All React components have explicit prop types — no `any`
 - ☐ All `useState` and `useRef` properly typed with generics
 - ☐ All event handlers typed with correct React event types
 - ☐ Generic custom hook implemented without `any`
 - ☐ Discriminated union used with exhaustive switch handling
 - ☐ Can explain `interface` vs `type` and `unknown` vs `any`
-

12. API Communication

Learning Objective

Learn to communicate with backend services using the Fetch API and Axios, handle errors gracefully, and understand REST conventions and HTTP status codes. By the end, you should be able to build a complete data-fetching layer for a React application.

Curriculum

1. [Fetch API — MDN](#)
 2. [Response.ok — MDN](#)
 3. [Axios — Getting Started](#)
 4. [Axios Interceptors](#)
 5. [HTTP Status Codes — MDN](#)
 6. [CORS — MDN](#)
-

Practice Exercises

Exercise 1: API Service Layer

Build a reusable API service module for a React app: - Create an Axios instance with `baseUrl`, `timeout`, and default headers - Add a request interceptor that attaches an auth token from `localStorage` - Add a response interceptor that handles 401 errors (redirect to login) - Implement functions: `getUsers()`, `getUserById(id)`, `createUser(data)`, `updateUser(id, data)`, `deleteUser(id)` - Each function must have proper error handling and return typed responses

Exercise 2: Full CRUD Interface

Build a React UI that performs all CRUD operations against `https://jsonplaceholder.typicode.com/`: - List users with a GET request on mount - Create a new user with a POST form — show the response in the UI - Update a user with a PUT request triggered by an “Edit” button - Delete a user with a DELETE request triggered by a “Delete” button with confirmation - Show loading states during requests and error messages on failure - Display relevant HTTP status codes in the UI for each operation

Exercise 3: Error Handling & Edge Cases

Build a demo page that demonstrates robust error handling: - Handle network failure (disconnect internet and attempt fetch) - Handle 404 response (fetch a non-existent resource) - Handle 500 response (use a mock/bad endpoint) - Handle request timeout (set a very short timeout) - Display user-friendly error messages for each scenario - Implement a retry button that re-attempts the failed request

Acceptance Criteria

- ☐ Axios instance created with interceptors for auth and error handling
 - ☐ All five CRUD operations working against a real API
 - ☐ Loading states shown during requests
 - ☐ Error messages displayed on failure with relevant status codes
 - ☐ Network failure, 404, 500, and timeout scenarios handled gracefully
 - ☐ Can explain the difference between Fetch API and Axios
 - ☐ Can list at least 10 HTTP status codes with their meanings
 - ☐ Can explain what CORS is and when it applies
-

13. CSS Frameworks & Preprocessors

Learning Objective

Learn to use Tailwind CSS for utility-first styling, SCSS/SASS for preprocessed CSS with variables and mixins, and CSS Modules for component-scoped styles. Understand when to use each approach in a project.

Curriculum

1. [Tailwind CSS — Installation](#)
 2. [Tailwind CSS — Utility-First Fundamentals](#)
 3. [Tailwind CSS — Responsive Design](#)
 4. [SASS/SCSS — Guide](#)
 5. [CSS Modules — GitHub](#)
-

Practice Exercises

Exercise 1: Rebuild a Component Three Ways

Take a single UI component (e.g., a pricing card) and build it three times using different approaches: - Version 1: Tailwind CSS — utility classes only, responsive with `md:` and `lg:` prefixes, dark mode with `dark:` - Version 2: SCSS — variables, nesting, a mixin for flex-center, BEM naming convention - Version 3: CSS Modules — `.module.css` file with scoped classes imported into a React component - Write a comparison document noting: development speed, readability, maintainability, and bundle size implications

Exercise 2: Tailwind CSS Landing Page

Build a complete responsive landing page using only Tailwind CSS: - Hero section with gradient background, responsive text sizes, and hover-animated CTA buttons - Feature grid using Tailwind's grid utilities (`grid-cols-1 md:grid-cols-3 gap-6`) - Dark mode support using `dark:` variants - Custom colors and fonts configured in `tailwind.config.js`

Exercise 3: SCSS Component Library

Build a small component library using SCSS: - Define variables for colors, spacing, typography, and breakpoints - Create mixins for: `flex-center` , `responsive-grid` , `button-variant($bg, $color)` - Build at least five styled components: button, card, badge, input, alert - Use BEM naming: `.card` , `.card__title` , `.card__body` , `.card--highlighted`

Acceptance Criteria

- ☐ Same component built with Tailwind, SCSS, and CSS Modules
 - ☐ Comparison document covers speed, readability, maintainability
 - ☐ Tailwind landing page is fully responsive with dark mode
 - ☐ Custom theme configured in `tailwind.config.js`
 - ☐ SCSS library uses variables, mixins, nesting, and BEM naming
 - ☐ Can explain when to choose Tailwind vs SCSS vs CSS Modules
-

14. Testing

Learning Objective

Learn the testing pyramid, unit testing with Vitest, component testing with React Testing Library, and end-to-end testing with Cypress. By the end, you should be able to write meaningful tests that give confidence in your code.

Curriculum

1. [Vitest — Getting Started](#)
 2. [React Testing Library — Introduction](#)
 3. [Testing Library — Queries](#)
 4. [Cypress — Getting Started](#)
 5. [Testing Overview — react.dev](#)
-

Practice Exercises

Exercise 1: Unit Testing with Vitest

Write comprehensive unit tests for a utility module: - Create a utility module with at least five functions (e.g., `capitalize`, `formatCurrency`, `validateEmail`, `slugify`, `debounce`) - Write tests using `describe`, `it`, `expect` — at least 3 test cases per function covering: normal input, edge cases, and error cases - Use `beforeEach` / `afterEach` for setup/teardown where needed - Mock a dependency using `vi.fn()` or `vi.mock()` in at least one test - Achieve 100% code coverage for the utility module

Exercise 2: React Component Testing

Write tests for three React components using React Testing Library: - Test a form component: render, fill inputs with `userEvent.type()`, submit with `userEvent.click()`, verify submission handler called with correct data - Test a list component: render with props, verify items displayed, test filter/search functionality - Test an async component: mock API call with `vi.fn()`, test loading state, test successful data display, test error state - Use `screen.getByRole()`, `screen.getByText()`, `screen.getByLabelText()` — avoid `getByTestId` where possible

Exercise 3: E2E Testing with Cypress

Write end-to-end tests for a complete user flow: - Install Cypress and write tests for a login flow: visit page, fill form, submit, verify redirect - Test a CRUD flow: create an item, verify it appears, edit it, delete it, verify removal - Use `data-testid` attributes for reliable element selection - Test at least one error scenario (invalid input, failed submission)

Acceptance Criteria

- ☐ Utility module has 100% test coverage with Vitest
 - ☐ At least 3 test cases per function (normal, edge, error)
 - ☐ `vi.fn()` or `vi.mock()` used for mocking
 - ☐ Three React components tested with React Testing Library
 - ☐ `userEvent` used for interactions (not `fireEvent`)
 - ☐ Async component tested with mocked API and loading/error states
 - ☐ Cypress E2E tests cover a complete user flow
 - ☐ Can explain the testing pyramid and when to use each level
 - ☐ Can explain the difference between unit, integration, and E2E tests
-

15. Performance Optimization

Learning Objective

Learn to measure and improve frontend performance using Core Web Vitals, code splitting, image optimization, memoization, and caching strategies. By the end, you should be able to identify bottlenecks and apply appropriate optimization techniques.

Curriculum

1. [Core Web Vitals — web.dev](#)
 2. [Code Splitting — react.dev](#)
 3. [Image Optimization — web.dev](#)
 4. [Performance Profiling — Chrome DevTools](#)
 5. [Lighthouse — web.dev](#)
-

Practice Exercises

Exercise 1: Performance Audit

Audit a real website (your own project or a public site) using Chrome DevTools and Lighthouse: - Run a Lighthouse audit and document scores for Performance, Accessibility, Best Practices, SEO - Identify the three largest resources in the Network tab waterfall - Identify render-blocking resources and suggest how to fix them - Measure LCP, INP, and CLS using the Performance tab - Write an optimization plan with at least five specific recommendations

Exercise 2: Implement Optimizations in a React App

Apply performance techniques to an existing React project: - Add code splitting with `React.lazy` + `Suspense` for at least three routes - Optimize images: add `loading="lazy"`, use `srcset` for responsive images, add `<picture>` with WebP/AVIF sources - Apply `React.memo` to at least two components and verify with Profiler - Implement debouncing on a search input (300ms delay before filtering) - Add `font-display: swap` to custom font loading - Run Lighthouse before and after — document the score improvement

Exercise 3: Bundle Analysis

Analyze and reduce bundle size: - Install and run a bundle analyzer (e.g., `rollup-plugin-visualizer` for Vite) - Identify the three largest dependencies in the bundle - Replace at least one heavy import with a lighter alternative or tree-shaken import - Verify the bundle size decreased after optimization

Acceptance Criteria

- ☐ Lighthouse audit completed with documented scores
- ☐ LCP, INP, CLS measured and documented
- ☐ Optimization plan with five specific recommendations
- ☐ Code splitting applied to at least three routes
- ☐ Images optimized with lazy loading, srcset, and modern formats
- ☐ `React.memo` applied and verified with DevTools Profiler
- ☐ Debouncing implemented on search input
- ☐ Lighthouse scores improved after optimizations
- ☐ Bundle analyzed and at least one dependency optimized
- ☐ Can explain Core Web Vitals and their target values

16. Web Security

Learning Objective

Understand common web vulnerabilities (XSS, CSRF) and security best practices for frontend development. By the end, you should be able to identify security risks in code and apply defensive measures.

Curriculum

1. [OWASP Top 10](#)
2. [XSS Prevention — OWASP](#)
3. [CSRF Prevention — OWASP](#)
4. [Content Security Policy — MDN](#)
5. [Web Security — MDN](#)

Practice Exercises

Exercise 1: Security Code Review

Review a provided (or self-written) “insecure” codebase and identify/fix vulnerabilities: - Find and fix at least one XSS vulnerability (e.g., `innerHTML` with user input) - Find and fix insecure token storage (e.g., token in `localStorage` instead of `httpOnly` cookie) - Find and fix a missing input validation/sanitization issue - Find and fix external links missing `rel="noopener noreferrer"` - Run `npm audit` and document any vulnerabilities found; fix critical ones

Exercise 2: Implement Security Best Practices

Add security measures to an existing React project: - Ensure no `dangerouslySetInnerHTML` is used without sanitization (use DOMPurify if needed) - Add input validation on all form fields (both client-side and explain server-side needs) - Implement a Content Security Policy meta tag - Store auth tokens in `httpOnly` cookies (explain why this is better than `localStorage`) - Add

`rel="noopener noreferrer"` to all external `<a>` links

Acceptance Criteria

- ☐ Can explain XSS and give two concrete examples of how it can occur
- ☐ Can explain CSRF and how anti-CSRF tokens prevent it
- ☐ Identified and fixed security vulnerabilities in a code review
- ☐ `npm audit` run and critical vulnerabilities addressed
- ☐ Input validation applied to all form fields
- ☐ Can explain why `httpOnly` cookies are more secure than `localStorage` for tokens
- ☐ CSP meta tag implemented
- ☐ All external links have `rel="noopener noreferrer"`

17. Accessibility (a11y)

Learning Objective

Learn WCAG principles and how to build accessible web applications using semantic HTML, ARIA attributes, keyboard navigation, and proper color contrast. By the end, you should be able to audit a webpage for accessibility and fix common issues.

Curriculum

1. [Web Accessibility Introduction — W3C WAI](#)
2. [WCAG Quick Reference](#)
3. [The A11Y Project Checklist](#)
4. [ARIA Authoring Practices — W3C](#)
5. [Accessibility — MDN](#)

Practice Exercises

Exercise 1: Accessibility Audit

Audit an existing project (or a public website) for accessibility: - Run Lighthouse accessibility audit and document the score - Test keyboard navigation: can you reach every interactive element with Tab? Can you activate buttons with Enter/Space? - Test with a screen reader (VoiceOver on Mac or NVDA on Windows) — document what is announced for the main navigation, headings, images, and form fields - Check color contrast ratios for all text using a contrast checker tool — flag any that fail WCAG AA (4.5:1 for normal text) - Create a report with at least five issues found and how to fix each

Exercise 2: Build an Accessible Form

Build a fully accessible contact form: - Every input has a visible `<label>` associated with `for / id` - Required fields marked with `aria-required="true"` and visual indicator - Error messages associated with inputs using `aria-describedby` - Form errors announced to screen readers using `aria-live="polite"` region - Keyboard-navigable: Tab through all fields, submit with Enter - Focus management: move focus to first error field on invalid submission - Color contrast passing WCAG AA for all text and UI elements

Exercise 3: Fix an Inaccessible Page

Take a page with common accessibility problems and fix them: - Add proper heading hierarchy (`h1` > `h2` > `h3` — no skipping levels) - Replace decorative images' `alt` text with empty `alt=""` and `role="presentation"` - Add `alt` text to all informational images - Add `aria-label` to icon-only buttons - Add skip navigation link (`<a href="#main-content">Skip to content</a>`) - Ensure all interactive elements have visible focus indicators - Test all fixes with keyboard navigation and screen reader

Acceptance Criteria

- ☐ Lighthouse accessibility audit completed with documented score
- ☐ Keyboard navigation tested — all interactive elements reachable
- ☐ Screen reader testing completed with documented findings
- ☐ Color contrast checked for all text — WCAG AA compliant
- ☐ Accessible form built with proper labels, error handling, and focus management
- ☐ `aria-required` , `aria-describedby` , `aria-live` used correctly
- ☐ Inaccessible page fixed with proper headings, alt text, focus indicators, and skip link
- ☐ Can explain the four WCAG principles (Perceivable, Operable, Understandable, Robust)

18. Progressive Web Apps (PWA)

Learning Objective

Learn what makes a Progressive Web App — installability, offline capability, and app-like experience. Understand service workers and web manifests. By the end, you should be able to add PWA capabilities to an existing web application.

Curriculum

1. [Learn PWA — web.dev](#)
2. [Web App Manifest — MDN](#)
3. [Service Workers — MDN](#)
4. [Offline Cookbook — web.dev](#)

Practice Exercises

Exercise 1: Add PWA Support to an Existing App

Convert an existing React/Vite project into a PWA: - Create a `manifest.json` with: `name` , `short_name` , `start_url` , `display: standalone` , `background_color` , `theme_color` , and at least two icon sizes (192x192, 512x512) - Link the manifest in `index.html` with `<link rel="manifest" href="/manifest.json">` - Register a basic service worker that caches the app shell (HTML, CSS, JS) on install - Implement a "cache-first" strategy for static assets and "network-first" for API calls - Test offline behavior: disconnect internet and verify the app still loads - Add an install prompt banner that appears when the browser detects the app is installable

Exercise 2: Offline-First Todo App

Build a simple todo app that works fully offline: - All data stored in `localStorage` or IndexedDB - Service worker caches all app assets - App loads instantly even with no network - When online, sync changes to a mock API endpoint - Show online/offline indicator in the UI using `navigator.onLine`

Acceptance Criteria

- ☐ `manifest.json` created with all required fields and icons
 - ☐ Service worker registered and caching app shell
 - ☐ Cache-first strategy for static assets, network-first for API calls
 - ☐ App loads and works offline (verified with DevTools Network: Offline)
 - ☐ Install prompt appears in the browser
 - ☐ Offline-first todo app works without network
 - ☐ Online/offline status indicator in the UI
 - ☐ Can explain what a service worker is and its lifecycle (install, activate, fetch)
-

19. DevTools & Debugging

Learning Objective

Master Chrome DevTools for inspecting elements, debugging JavaScript, monitoring network requests, profiling performance, and auditing with Lighthouse. Learn to use React DevTools for component and performance inspection.

Curriculum

1. [Chrome DevTools Overview](#)
 2. [Elements Panel](#)
 3. [Console Panel](#)
 4. [Sources Panel & Debugging](#)
 5. [Network Panel](#)
 6. [Performance Panel](#)
 7. [React DevTools — Chrome Extension](#)
-

Practice Exercises

Exercise 1: DevTools Deep Dive

Complete a hands-on DevTools exploration on any website: - **Elements:** Inspect and live-edit the CSS of at least three elements; take before/after screenshots - **Console:** Use `console.log`, `console.table`, `console.time` / `console.timeEnd`, `console.group` / `console.groupEnd` to debug a script - **Network:** Record a page load, identify the slowest resource, check if resources are cached (look for 304 responses), examine request/response headers - **Sources:** Set a breakpoint in JavaScript code, step through execution line by line, inspect variable values in the Scope panel - **Performance:** Record a page interaction, identify the longest task in the flamegraph, document what caused it - **Application:** Inspect `localStorage`, `sessionStorage`, and cookies for the current site

Exercise 2: Debug a Broken React App

Given (or create) a React app with five intentional bugs, use DevTools to find and fix each: - A component not rendering (check the Components tab for props/state) - An infinite re-render loop (check the Console for repeated logs, use Profiler) - A failed API call (check Network tab for status code and response body) - A CSS layout issue (inspect with Elements, toggle CSS properties to find the cause) - A state update not reflecting in UI (use React DevTools to inspect component state) - Document each bug: what you saw, how you found it, and how you fixed it

Acceptance Criteria

- ☐ Demonstrated live CSS editing in Elements panel
 - ☐ Used at least four `console` methods beyond `console.log`
 - ☐ Network tab used to identify slowest resource and inspect headers
 - ☐ Breakpoint set and code stepped through in Sources panel
 - ☐ Performance recording analyzed with longest task identified
 - ☐ `localStorage/sessionStorage/cookies` inspected in Application panel
 - ☐ Five bugs found and fixed using DevTools — each documented
 - ☐ React DevTools used to inspect component tree, props, state, and hooks
 - ☐ Can explain the difference between the Performance tab and Lighthouse
-

20. Deployment & CI/CD

Learning Objective

Learn how to deploy frontend applications to modern hosting platforms (Vercel, Netlify) and set up continuous integration/deployment with GitHub Actions. By the end, you should be able to deploy a project with automated testing on every push.

Curriculum

1. [Vercel — Getting Started](#)
 2. [Netlify — Getting Started](#)
 3. [GitHub Actions — Quickstart](#)
 4. [GitHub Actions — Workflow Syntax](#)
 5. [Environment Variables — Vercel](#)
-

Practice Exercises

Exercise 1: Deploy to Vercel and Netlify

Deploy the same React project to both platforms and compare: - Deploy to Vercel using `vercel` CLI — configure a custom domain or subdomain if available - Deploy to Netlify using `netlify deploy --prod` — set up build settings (build command, publish directory) - Set up an environment variable on each platform (e.g., `VITE_API_URL`) and access it in the app - Push a code change and verify both platforms auto-deploy from the Git repository - Document the differences: ease of setup, build speed, preview deploys, dashboard features

Exercise 2: GitHub Actions CI/CD Pipeline

Set up a complete CI/CD workflow: - Create `.github/workflows/ci.yml` that runs on push to `main` and on all pull requests - Steps: checkout code, install Node.js, install dependencies (`npm ci`), run linter, run tests, run build - Add a badge to your `README.md` showing the CI status - Create a failing test intentionally and verify the CI pipeline catches it (shows red) - Fix the test and verify the pipeline passes (shows green)

Exercise 3: Preview Deployments

Set up preview deployments for pull requests: - Configure Vercel or Netlify to create a unique preview URL for every PR - Open a PR with a UI change and share the preview URL - Verify the preview shows the changes without affecting the production deployment

Acceptance Criteria

- ☐ Project deployed and live on Vercel
 - ☐ Project deployed and live on Netlify
 - ☐ Environment variables configured on both platforms
 - ☐ Auto-deploy from Git working on both platforms
 - ☐ GitHub Actions CI pipeline running on push and PRs
 - ☐ Pipeline runs: lint, test, build — in correct order
 - ☐ CI badge added to README.md
 - ☐ Failing test caught by CI (verified with screenshot)
 - ☐ Preview deployment URL generated for a PR
 - ☐ Can explain the difference between CI and CD
 - ☐ Can explain what `npm ci` does differently from `npm install`
-

21. Soft Skills & Best Practices

Learning Objective

Develop professional habits around code quality, code review, and continuous learning that separate junior developers from those who grow quickly. These skills are as important as technical knowledge for a successful career.

Curriculum

1. [Clean Code JavaScript — GitHub](#)
 2. [Google Engineering Practices — Code Review](#)
 3. [Conventional Commits](#)
 4. [How to Write Good PR Descriptions](#)
-

Practice Exercises

Exercise 1: Code Review Practice

Review three pull requests (from teammates, open source, or self-created): - For each PR, write at least three constructive review comments - Each comment must: identify the specific issue, explain why it matters, and suggest a concrete fix - Practice both “requesting changes” and “approving with suggestions” - Write a self-review of your own most recent PR using the same standards

Exercise 2: Refactor Messy Code

Take a 50-100 line block of “working but messy” code and refactor it: - Apply the DRY principle — extract repeated logic into reusable functions - Apply single responsibility — ensure each function does one thing - Improve naming — replace vague names (`data`, `temp`, `x`) with descriptive ones - Remove dead code, unnecessary comments, and `console.logs` - Write a before/after comparison explaining each change

Exercise 3: Daily Practice Plan

Create a personal 4-week learning plan for after training completes: - Week 1-2: Build a personal project from scratch using the full stack learned - Week 3: Contribute to an open-source project (find a "good first issue") - Week 4: Study one advanced topic in depth (e.g., Next.js, WebSockets, animations) - Include specific daily goals (e.g., "Monday: set up project, Tuesday: build API layer") - List five resources (blogs, newsletters, YouTube channels) to follow for staying updated

Acceptance Criteria

- ☐ Three PRs reviewed with constructive, specific feedback
- ☐ Self-review completed on own PR
- ☐ Messy code refactored with before/after comparison
- ☐ DRY, single responsibility, and clean naming applied
- ☐ 4-week personal learning plan created with daily goals
- ☐ Five learning resources identified for ongoing development
- ☐ Can explain DRY, single responsibility, and why clean code matters
- ☐ Can give constructive code review feedback that is specific and actionable

Training Timeline (Suggested)

Week	Focus Area
1-2	HTML & CSS fundamentals, responsive design
3-4	JavaScript core + advanced concepts
5	Git, npm, build tools
6-7	React fundamentals + hooks
8	React Router, Context API, state management
9	TypeScript basics with React
10	API integration, TanStack Query
11	Testing (unit + integration)
12	Performance, accessibility, deployment

Remember: The best frontend developer is not the one who memorizes everything, but the one who knows how to find answers, debug effectively, and build real projects.

Last Updated: April 2026